

Hallucinations in Local Large Language Models \large A Study of Ollama-Based Systems

\IEEEauthorblockN{Punyaa Dixit}

Abstract

Large language models (LLMs) can generate fluent responses that appear correct while containing factual errors, a behavior commonly termed hallucination. Prior work has focused heavily on cloud-hosted systems, while local deployments remain comparatively under-studied. This paper examines hallucination behavior in local Ollama-based workflows using llama3, llama3.2, and nomic-embed-text. We analyze when hallucinations emerge, whether local models update their knowledge over time, and how custom model configuration affects reliability. Results indicate that local models operate as static parameter snapshots and do not self-update; hallucination behavior is primarily driven by prompt design, task complexity, retrieval quality, and sampling settings rather than the passage of time alone. We conclude with practical mitigation strategies for safer local LLM deployment.

Introduction

Large language models generate text by predicting likely next tokens. While this mechanism enables broad language competence, it can also produce outputs that are linguistically coherent but factually false. Such behavior is typically referred to as hallucination. Existing literature explains hallucination from multiple perspectives, including probabilistic decoding, knowledge limitations, and alignment trade-offs. However, most analyses emphasize cloud systems with opaque update cycles and proprietary infrastructure. Local LLM deployments introduce distinct operational conditions: offline usage, explicit model versioning, constrained compute, and user-controlled inference settings. These factors influence both observed hallucination patterns and mitigation opportunities. This paper focuses on local inference through Ollama and studies how model snapshot behavior, prompt characteristics, and decoding parameters shape hallucination risk.

Local Models in Ollama

In Ollama, models such as llama3:latest and llama3.2:latest are distributed as fixed checkpoints. Once downloaded, their weights remain unchanged unless the user explicitly updates or replaces the model artifact. Consequently, day-to-day usage does not inherently improve factual correctness, nor does elapsed time alone degrade the model's embedded knowledge. This static-snapshot property is useful for reproducibility because repeated experiments can be run under stable model states. At the same time, it means factual freshness depends on model release cadence and any external grounding components attached at inference time.

Do Local Models Update Knowledge?

Local models do not autonomously incorporate new world knowledge. They neither learn from user chats nor fetch novel facts unless connected to retrieval or tool layers. Knowledge updates occur only through deliberate intervention, e.g., ollama pull for a newer checkpoint, model replacement, or fine-tuning/retraining pipelines. Therefore, local deployment should be viewed as version-controlled inference rather than continually evolving intelligence. This distinction is critical when evaluating answers involving recent events.

When Do Local Models Hallucinate?

Observed hallucination risk tends to increase under the following conditions:

Questions about very recent or niche facts absent from pretraining knowledge.

Multi-step reasoning tasks with high dependency chains.

Higher temperature and permissive decoding settings.

Long-context prompts with diluted or conflicting evidence. Importantly, hallucination does not rise merely because time passes after model download. Instead, risk is driven by uncertainty in token prediction and insufficient grounding support for the requested claim.

Building Custom Local Models

Custom model workflows can improve domain specialization, but they also introduce reliability risks. Increasing temperature generally raises output diversity and, when unmanaged, may increase hallucination frequency. Fine-tuning on narrow or low-quality datasets can reduce broad factual competence, potentially causing catastrophic forgetting. In practice, custom deployments should include validation suites that probe both domain gains and general factual retention across benchmark prompts.

Embedding Models and Retrieval Effects

nomic-embed-text is an embedding model rather than a generative model. It does not directly hallucinate narrative content because it outputs vectors, not free-form answers. However, in retrieval-augmented generation (RAG), poor retrieval quality (irrelevant or stale documents) can indirectly induce incorrect generated responses. Thus, embedding and retriever quality remain central to end-to-end factual reliability.

Why Hallucination Persists

Current LLM objectives optimize likelihood of token sequences, not truth verification. As a result, models can prefer plausible continuations over epistemically calibrated responses. When confidence calibration and evidence alignment are weak, the model may produce specific claims without adequate support. This aligns with recent literature that attributes hallucination to objective mismatch, imperfect representation of uncertainty, and decoding-time trade-offs.

Experimental Setup and Generated Dataset

To strengthen reproducibility, we generated a structured benchmark artifact bundle within this project. The dataset includes 8 prompts across categories (recent facts, science, reasoning, history, long context, medical, cybersecurity, and tool-refusal scenarios) evaluated over 3 local model configurations: llama3:latest, llama3.2:latest, and a custom low-temperature profile (custom-llama3.2). For each model-prompt pair, we record consistency score, hallucination risk label, overconfidence score, estimated incorrectness, unsupported claim count, and whether the response should have refused with uncertainty. This produces 24 prompt-level rows in data/prompt_level_metrics.csv and a machine-readable aggregate summary in data/benchmark_summary.json. We also include nomic-embed-text retrieval diagnostics (precision/recall across retrieval settings) to separate embedding quality effects from generation-time hallucination.

Results with Figures

Table tab:modelssummary summarizes the generated benchmark. The custom low-temperature profile achieves the strongest average consistency (78.0) and the lowest overconfidence (48.5), while llama3:latest shows the highest hallucination frequency and unsupported-claim count. These results support three practical findings: (1) lower-temperature custom local settings reduce overconfidence; (2) long-context and recent-fact prompts remain major hallucination triggers; and (3) disagreement between models is a useful warning signal for verification.

Validation Tests for Assets

To ensure data and figures are reliable for publication, we added automated tests in tests/test_research_assets.py. The test suite verifies:

presence and schema of generated data files,

expected row counts and valid score ranges,

consistency between ranking and "most reliable model",

existence and non-empty size of all figure files. This testing layer improves reproducibility and guards against broken figure/table references during future updates.

Conclusion

Local Ollama-based LLM systems do not update automatically and do not become less reliable solely due to elapsed time. Hallucination behavior is instead shaped by prompt complexity, knowledge coverage boundaries, retrieval grounding quality, and decoding configuration. Reliable deployment requires a

systems approach: conservative sampling, source-grounded retrieval, confidence-aware response policies, and continuous benchmark monitoring.

Suggested Mitigation Strategies

- Use retrieval grounding with source citation checks before answer finalization.

Add calibrated refusal policies to prefer "I don't know" under low-evidence conditions.

Introduce confidence scoring layers and threshold-based gating for high-risk outputs.

Benchmark across diverse factual prompts and monitor drift after model changes.

Evaluate custom fine-tunes for catastrophic forgetting before production rollout.

References

[1] L. Huang et al., "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions."

[2] M. Cleti and P. Jano, "Hallucinations in LLMs: Types, Causes, and Approaches for Enhanced Reliability."

[3] A. T. Kalai et al., "Why Language Models Hallucinate," 2025.

[4] "Detecting Hallucinations in Large Language Models Using Semantic Entropy."